

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Вычислительная математика»
Тема: Метод Ньютона

Студент гр. 4342

Озернов Е.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Цель лабораторной работы — изучить численное решение нелинейного уравнения $f(x) = 0$ методом Ньютона. В ходе работы требуется отделить корень, реализовать подпрограммы вычисления $f(x)$ и $f'(x)$ и алгоритм NEWTON, а также выбрать начальное приближение x_0 на отрезке $[Left; Right]$, обеспечивающее сходимость метода. Также необходимо исследовать скорость сходимости при изменении Eps в диапазоне от 0.1 до 0.000001 и оценить обусловленность метода, моделируя округление значений $f(x)$ и $f'(x)$ с параметром Delta с помощью Round.

Задание

В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$, сходящейся к корню уравнения $f(x) = 0$. Достаточные условия сходимости метода формулируются теоремой, приведенной в [1,7].

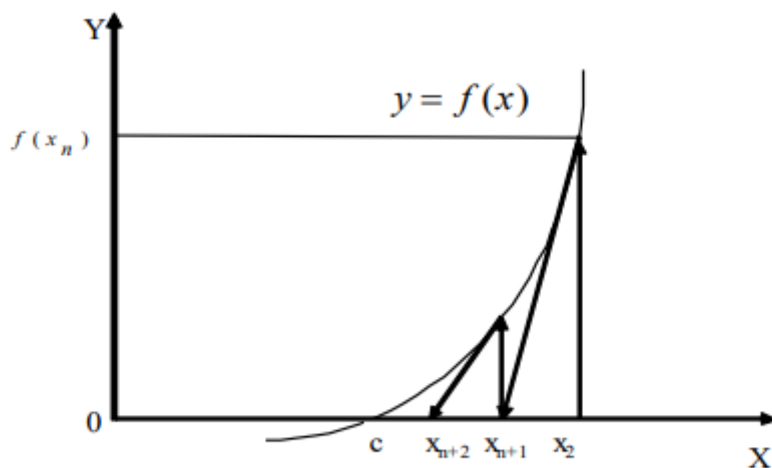


Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью Ox будет очередным приближением x_{n+1} корня уравнения $f(x) = 0$.

Для оценки погрешности n -го приближения корня предлагается пользоваться неравенством: $|c - x_n| \leq (M_2 / (2m_1)) * (x_n - x_{n-1})^2$, где M_2 — наибольшее значение модуля второй производной $|f''(x)|$ на отрезке $[a; b]$; m_1 — наименьшее значение модуля первой производной $|f'(x)|$ на отрезке $[a; b]$. Таким образом, если $|x_n - x_{n-1}| < \epsilon$, то $|c - x_n| \leq (M_2 / (2m_1)) * \epsilon^2$.

Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, то есть процесс сходится очень быстро (имеет место квадратическая сходимостъ). Из указанного следует, что при необходимости нахождения корня с точностью ϵ ps итерационный процесс можно прекращать, когда $|x_n - x_{n-1}| < \epsilon_{ps}0$, где $\epsilon_{ps}0 = (2 \cdot m1 / M2) \cdot \epsilon$ ps. (3.1)

Рассмотрим один шаг итераций. Если на $(n-1)$ -м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1})$ и $f'(x_{n-1})$, и следующее приближение корня определяется по формуле: $x_n = x_{n-1} - f(x_{n-1}) / f'(x_{n-1})$.

При выполнении условия (3.1) величина x_n принимается за приближенное значение корня s , вычисленное с точностью ϵ ps.

В лабораторной работе №5 предлагается, используя программы-функции NEWTON и ROUND из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x) = 0$ с заданной точностью ϵ ps методом Ньютона, исследовать скорость сходимости и обусловленность метода. Для данной работы вид функции $f(x)$ задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №5:

1. Графически или аналитически отделить корень уравнения $f(x) = 0$ (то есть найти отрезки $[Left; Right]$, на котором функция $f(x)$ удовлетворяет условиям сходимости метода Ньютона).
2. Составить подпрограммы-функции вычисления $f(x)$ и $f'(x)$, предусмотрев округление их значений с заданной точностью Δ ta.
3. Составить головную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограммам $f(x)$, $f'(x)$, Round, NEWTON и индикацию результатов.

4. Выбрать начальное приближение корня x_0 из $[Left; Right]$ так, чтобы выполнялось условие: $f(x_0) * f''(x_0) > 0$.
5. Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Вариант 16: $f(x) = \arcsin(2x/(1+x^2)) - e^{(-x^2)}$

Выполнение работы

Отделение (изоляция) корня уравнения $f(x) = 0$ и проверка условий сходимости метода Ньютона

Рассматривается функция:

$$f(x) = \arcsin(2x / (1 + x^2)) - e^{(-x^2)}.$$

Функция $f(x)$ непрерывна на $\mathbb{R}$, так как выражение $2x / (1 + x^2)$ при любых x принадлежит отрезку $[-1; 1]$, поэтому $\arcsin(\cdot)$ определена для всех x , а функция $e^{(-x^2)}$ также непрерывна. Следовательно, $f(x)$ непрерывна на любом отрезке $[Left; Right]$.

Для отделения корня используем теорему Коши (Больцано): если $f(x)$ непрерывна на $[Left; Right]$ и выполняется условие $f(Left) * f(Right) < 0$,

то на интервале $(Left; Right)$ существует хотя бы один корень уравнения $f(x) = 0$.

Проверим значения функции на концах интервала:

$$\text{При } x = 0: f(0) = \arcsin(0) - e^0 = 0 - 1 = -1 < 0.$$

$$\text{При } x = 0.5: f(0.5) = \arcsin(2*0.5 / (1 + 0.5^2)) - e^{(-0.5^2)} = \arcsin(1 / 1.25) - e^{(-0.25)} = \arcsin(0.8) - e^{(-0.25)} \approx 0.9273 - 0.7788 \approx 0.1485 > 0.$$

Так как $f(0) < 0$ и $f(0.5) > 0$, то: $f(0) * f(0.5) < 0$.

Следовательно, на отрезке $[0; 0.5]$ существует корень уравнения $f(x) = 0$. Этот отрезок принимаем в качестве интервала отделения корня: $[Left; Right] = [0; 0.5]$.

Дополнительно для применимости и устойчивой сходимости метода Ньютона важно, чтобы на выбранном интервале не обращалась в ноль производная $f'(x)$ и чтобы можно было выбрать начальное приближение x_0 , удовлетворяющее условию: $f(x_0) * f''(x_0) > 0$.

На практике это условие проверяется для точек из интервала $[Left; Right]$ (например, для концов отрезка и середины), после чего выбирается подходящее начальное приближение x_0 для запуска итерационного процесса Ньютона.

2. Подпрограммы вычисления $f(x)$ и $f'(x)$ с учётом округления Round (Delta)

Для реализации метода Ньютона требуется составить подпрограммы (функции), вычисляющие значение функции $f(x)$ и её первой производной $f'(x)$ в произвольной точке x . Эти подпрограммы используются на каждой итерации метода Ньютона, так как очередное приближение вычисляется по формуле: $x_{n+1} = x_n - f(x_n) / f'(x_n)$.

В данной лабораторной работе дополнительно необходимо предусмотреть моделирование ошибок исходных данных и вычислений, то есть округление значений $f(x)$ и $f'(x)$ с заданной точностью Delta с использованием подпрограммы Round.

2.1. Подпрограмма вычисления функции $f(x)$

Функция для моего варианта задаётся формулой: $f(x) = \arcsin(2x / (1 + x^2)) - e^{-x^2}$.

Где:

x — вещественный аргумент;

$\arcsin(z)$ — арксинус (обратная функция к $\sin$), определённый для $z \in [-1; 1]$;

e^{-x^2} — экспоненциальная функция.

Область определения и корректность вычисления

Выражение $2x / (1 + x^2)$ при любых $x \in \mathbb{R}$ принадлежит отрезку $[-1; 1]$, следовательно $\arcsin(2x / (1 + x^2))$ определён для всех вещественных x . Значение e^{-x^2} также определено и непрерывно для любых x . Поэтому $f(x)$ непрерывна на всей числовой прямой.

Особенность машинных вычислений

При вычислениях с типом `double` возможны округления, из-за которых величина $t = 2x / (1 + x^2)$ может получиться немного больше 1 или меньше -1 (например, 1.0000000001). Это может вызвать ошибку при вычислении $\arcsin(t)$. Чтобы гарантировать корректность, аргумент арксинуса ограничивают диапазоном $[-1; 1]$: если $t > 1$, то $t = 1$; если $t < -1$, то $t = -1$.

Таким образом, вычисление $f(x)$ выполняется по схеме:

$t = 2x / (1 + x^2);$

$t = \text{clamp}(t, -1, 1);$

$f(x) = \text{asin}(t) - \exp(-x^2).$

Пример реализации подпрограммы $f(x)$ на C++:

```
double f(double x)
{
    double t = (2x) / (1 + xx);
    if (t > 1) t = 1;
    if (t < -1) t = -1;
    return asin(t) - exp(-x*x);
}
```

2.2. Подпрограмма вычисления первой производной $f'(x)$

Для метода Ньютона требуется также вычислять $f'(x)$. Производная находится аналитически. Обозначим: $t(x) = 2x / (1 + x^2)$.

Тогда: $f(x) = \arcsin(t(x)) - e^{(-x^2)}$.

Производная $\arcsin(t)$ равна: $d/dx[\arcsin(t(x))] = t'(x) / \sqrt{1 - t(x)^2}$.

Производная $t(x)$ вычисляется по правилу дроби: $t'(x) = 2*(1 - x^2) / (1 + x^2)^2$.

Производная второго слагаемого: $d/dx[-e^{(-x^2)}] = 2x * e^{(-x^2)}$.

Итоговая формула для первой производной:

$f'(x) = t'(x) / \sqrt{1 - t(x)^2} + 2x * e^{(-x^2)},$

где $t(x) = 2x / (1 + x^2)$, $t'(x) = 2*(1 - x^2) / (1 + x^2)^2$.

При реализации также учитывается, что $\sqrt{1 - t(x)^2}$ может стать очень малым, если $t(x)$ близко к ± 1 . Поэтому в программе используется защита от деления на очень маленькое число.

Пример реализации подпрограммы $f'(x)$ на C++:

```
double fp(double x)
{
    double denom = 1 + xx;
```



```

double t = (2x) / denom;
if (t > 1) t = 1;
if (t < -1) t = -1;
double tp = 2*(1 - xx) / (denomdenom);
double s = sqrt( max(0, 1 - tt) );
double part1 = (s < 1e-15) ? 0 : tp / s;
double part2 = 2xexp(-xx);
return part1 + part2;
}

```

2.3. Округление значений с точностью Delta (Round)

Чтобы исследовать чувствительность метода Ньютона к ошибкам в исходных данных, в работе применяется округление значений функции и производной: $f_Delta(x) = Round(f(x), Delta)$, $f'_Delta(x) = Round(f'(x), Delta)$.

Подпрограмма Round округляет число v к ближайшему значению, кратному Delta. Если $Delta = 0$, округление не применяется и используются точные значения: $f_Delta(x) = f(x)$, $f'_Delta(x) = f'(x)$.

В ходе итерационного процесса Ньютона вместо $f(x_n)$ и $f'(x_n)$ используются округлённые значения $f_Delta(x_n)$ и $f'_Delta(x_n)$, что позволяет оценить влияние параметра Delta на точность найденного корня и на число итераций.

3. Головная программа: обращение к $f(x)$, $f'(x)$, Round, NEWTON и индикация результатов

Для выполнения лабораторной работы составляется головная программа (main), которая организует полный вычислительный процесс методом Ньютона: задаёт начальные параметры задачи, обращается к подпрограммам вычисления функции $f(x)$ и её производной $f'(x)$, вызывает процедуру NEWTON, при необходимости моделирует ошибки исходных данных с помощью Round и выводит результаты на экран и в файлы.

Структура головной программы включает следующие этапы.

1. Задание исходных данных и параметров расчёта.

В программе задаются:

- функция $f(x)$ в виде отдельной подпрограммы:

$$f(x) = \arcsin(2x / (1 + x^2)) - e^{-x^2};$$

- подпрограмма вычисления производной $f'(x)$:

$$f'(x) = t'(x) / \sqrt{1 - t(x)^2} + 2x * e^{-x^2},$$

$$\text{где } t(x) = 2x / (1 + x^2), t'(x) = 2 * (1 - x^2) / (1 + x^2)^2;$$

- отрезок отделения корня, полученный в пункте 1:

$$[Left; Right] = [0; 0.5];$$

- начальное приближение x_0 , выбираемое из $[Left; Right]$ (в пункте 4);

• точность метода Eps (в дальнейшем она изменяется в диапазоне от 0.1 до 0.000001);

- ограничение на число итераций n_{max} (защита от отсутствия сходимости);

• параметр $Delta$ для моделирования ошибок исходных данных (округление значений f и f').

2. Обращение к подпрограммам $f(x)$ и $f'(x)$.

Головная программа передаёт аргумент x в подпрограммы $f(x)$ и $f'(x)$ и получает значения функции и производной. Это необходимо:

- для проверки условия отделения корня (знаки $f(Left)$ и $f(Right)$);
- для работы метода Ньютона, который на каждой итерации вычисляет

$f(x_n)$ и $f'(x_n)$.

3. Вызов процедуры $Round$ для моделирования ошибок (обусловленность).

Для исследования чувствительности метода Ньютона к ошибкам исходных данных используется округление значений функции и производной:

$$f_Delta(x) = Round(f(x), Delta),$$

$$f'_Delta(x) = Round(f'(x), Delta).$$

Смысл $Round$ состоит в имитации ограниченной точности вычислений. В программе $Round$ применяется при вычислении $f(x_n)$ и $f'(x_n)$, чтобы итерационный процесс Ньютона выполнялся уже с “искажёнными” значениями. Если $Delta = 0$, округление не выполняется и используются точные значения:

$$f_Delta(x) = f(x),$$

$$f'_Delta(x) = f'(x).$$

4. Вызов процедуры NEWTON для нахождения корня.

Основной расчёт выполняется методом Ньютона. На вход в NEWTON передаются:

- ссылка на функцию $f(x)$;
- ссылка на производную $f'(x)$;
- границы интервала [Left; Right];
- начальное приближение x_0 ;
- требуемая точность Eps;
- ограничение nmax;
- параметр Delta для Round (0 — без округления).

На каждой итерации вычисляется новое приближение корня по формуле:

$$x_{n+1} = x_n - f(x_n) / f'(x_n).$$

Условие остановки итерационного процесса в программе основано на малости изменения приближения:

$$|x_{\{n\}} - x_{\{n-1\}}| < eps0,$$

$$\text{где } eps0 = (2 * m1 / M2) * Eps,$$

$$m1 = \min_{\{x \text{ in } [Left; Right]\}} |f'(x)|,$$

$$M2 = \max_{\{x \text{ in } [Left; Right]\}} |f''(x)|.$$

Дополнительно может контролироваться условие:

$$|f(x_n)| < Eps$$

(как практический критерий малости значения функции).

Процедура NEWTON возвращает:

- найденное приближение корня x^* ;
- число итераций Iterations;
- величину последнего шага $|x_n - x_{\{n-1\}}|$;
- признак успешной сходимости (ok).

5. Индикация (вывод) результатов.

Головная программа выводит результаты в двух формах.

5.1 Вывод на экран (консоль):

- найденный интервал отделения корня [Left; Right];
- значения $f(\text{Left})$ и $f(\text{Right})$ (контроль смены знака);
- выбранное начальное приближение x_0 ;
- найденное значение корня x^* ;
- значение $f(x^*)$ (контроль близости к нулю);
- число итераций Iterations;
- последний шаг $|x_n - x_{n-1}|$.

5.2 Вывод в файлы для построения графиков:

Файл зависимости числа итераций от точности:

iterations_vs_eps_newton.csv

со столбцами: Eps; Iterations; Root; LastStep,

где LastStep = $|x_n - x_{n-1}|$.

Файл для исследования чувствительности к Delta:

sensitivity_vs_delta_newton.csv

со столбцами: Delta; Eps; Iterations; Root; AbsErrorToRef,

где AbsErrorToRef = $|x_{\text{Delta}} - x_{\text{ref}}|$,

x_{ref} — “опорное” значение корня, найденное при $\Delta = 0$ с высокой точностью.

Итог.

Таким образом, головная программа объединяет все компоненты лабораторной работы: вычисление $f(x)$ и $f'(x)$, поиск корня методом Ньютона (NEWTON), моделирование ошибок округления (Round), а также вывод результатов для анализа сходимости (зависимость Iterations от Eps) и обусловленности (влияние Delta на найденный корень).

4. Выбор начального приближения x_0 из $[Left; Right]$ по условию $f(x_0)$

$$* f'(x_0) > 0$$

Метод Ньютона обладает высокой скоростью сходимости, однако его успешная работа существенно зависит от выбора начального приближения x_0 . Для обеспечения сходимости итерационного процесса в лабораторной работе используется достаточное условие выбора начальной точки: $f(x_0) * f'(x_0) > 0$, где $f'(x_0)$ — вторая производная функции в точке x_0 .

Смысл данного условия заключается в следующем. Если выбрать x_0 так, что $f(x_0)$ и $f'(x_0)$ имеют одинаковый знак, то касательная к графику функции в точке $(x_0; f(x_0))$ пересечёт ось Ox “с правильной стороны” и последовательность приближений $x_1, x_2, \dots$ будет монотонно приближаться к корню, не уходя из области от деления корня. Это уменьшает риск расходимости и обеспечивает устойчивую сходимость метода.

Выбор x_0 выполняется внутри интервала от деления корня: $x_0 \in [Left; Right]$.

На практике значение $f'(x)$ удобно получать численно (приближённо), например методом центральных разностей: $f'(x) \approx (f(x + h) - f(x - h)) / (2h)$, где h — малый шаг (например, $h = 10^{-5}$ или $h = 10^{-5} * \max(1, |x|)$).

Далее проверяется условие $f(x) * f'(x) > 0$ для нескольких точек на интервале, например: для Left: $f(Left) * f'(Left)$, для Right: $f(Right) * f'(Right)$, для середины: $f((Left + Right)/2) * f'((Left + Right)/2)$.

В качестве начального приближения x_0 выбирается та точка из $[Left; Right]$, для которой условие $f(x_0) * f'(x_0) > 0$ выполняется. Если условие выполняется для нескольких точек, допускается выбрать, например, одну из границ или середину интервала, чтобы обеспечить более быстрое сближение к корню.

Таким образом, начальное приближение x_0 выбирается не произвольно, а по признаку, гарантирующему устойчивую работу метода Ньютона и его быструю (квадратическую) сходимость.

5. Проведение вычислений. Исследование скорости сходимости и чувствительности метода Ньютона к ошибкам исходных данных

После реализации подпрограмм $f(x)$, $f'(x)$, процедур Round и NEWTON выполняются вычисления по программе для исследования двух характеристик метода Ньютона: скорости сходимости и чувствительности к ошибкам в исходных данных. Расчёты проводятся на интервале отделения корня $[Left; Right] = [0; 0.5]$, при выбранном начальном приближении x_0 , удовлетворяющем условию $f(x_0) * f'(x_0) > 0$.

5.1. Исследование скорости сходимости метода Ньютона

Для исследования скорости сходимости выполняется серия запусков метода NEWTON при различных значениях точности Eps в диапазоне: $Eps \in [0.1; 0.000001]$.

На каждой итерации метода Ньютона вычисляется очередное приближение по формуле: $x_{n+1} = x_n - f(x_n) / f'(x_n)$.

В программе используется критерий останова по малости изменения приближений: $|x_n - x_{n-1}| < eps_0$, где $eps_0 = (2 * m_1 / M_2) * Eps$.

Дополнительно может контролироваться условие малости значения функции: $|f(x_n)| < Eps$.

Для каждого значения Eps фиксируются:

- число итераций Iterations, необходимое для достижения требуемой точности;
- найденное приближение корня Root;
- величина последнего шага LastStep = $|x_n - x_{n-1}|$.

Результаты автоматически сохраняются в файл:

iterations_vs_eps_newton.csv со столбцами: Eps; Iterations; Root; LastStep.

Далее по данным из файла строится график зависимости числа итераций от точности Eps, который позволяет экспериментально оценить скорость сходимости метода Ньютона. Метод Ньютона при корректном выборе начального приближения характеризуется квадратической сходимостью: при успешной работе метода число верных десятичных знаков в приближении

обычно быстро увеличивается, поэтому требуемая точность достигается за малое число итераций.

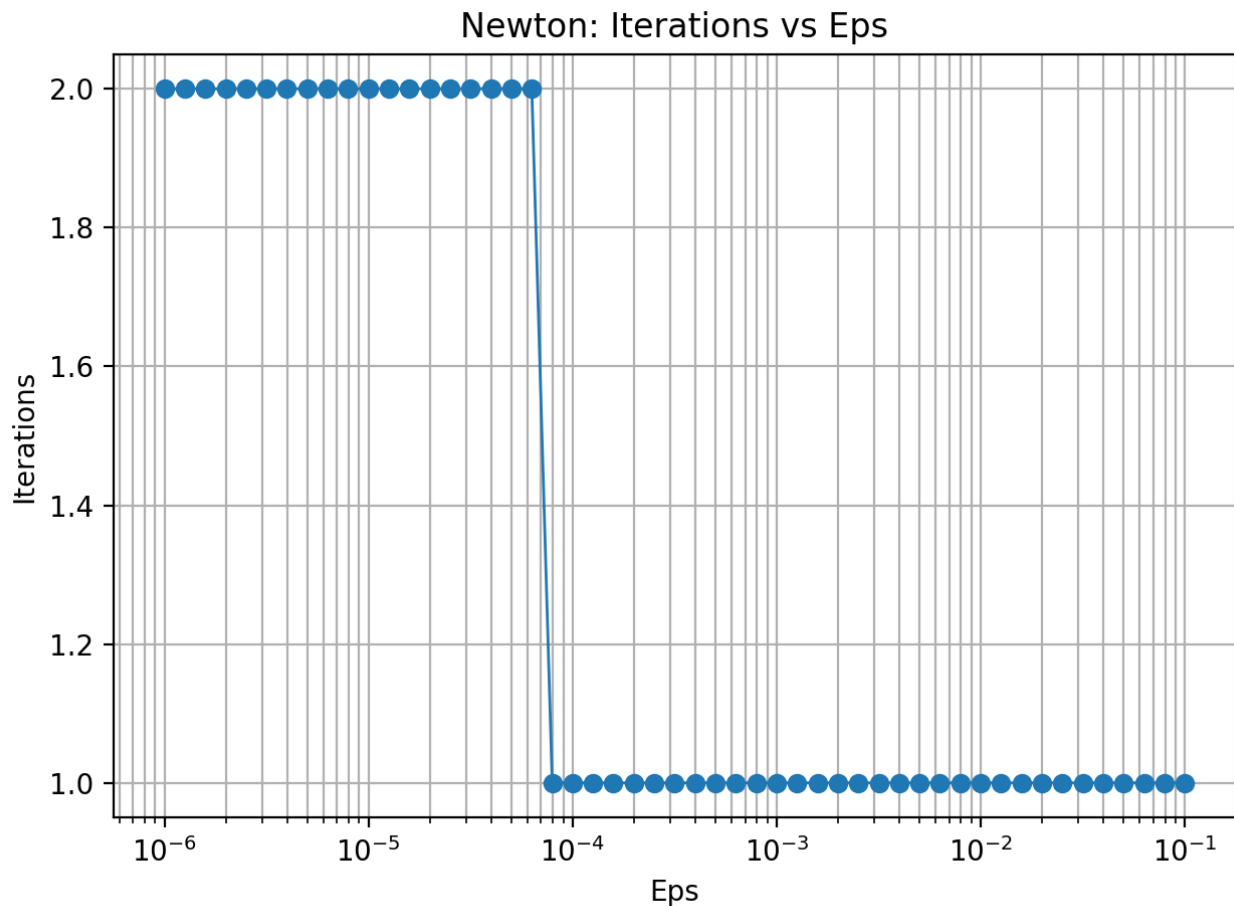


Рисунок 1 — График зависимости числа итераций метода Ньютона от точности Eps.

5.2. Исследование чувствительности метода Ньютона к ошибкам в исходных данных (обусловленность)

Для исследования чувствительности метода к ошибкам вычислений моделируются погрешности в значениях функции и производной с помощью округления Round с параметром Delta: $f_Delta(x) = \text{Round}(f(x), \text{Delta})$, $f'_Delta(x) = \text{Round}(f'(x), \text{Delta})$.

При $\text{Delta} = 0$ округление не применяется и метод работает с точными значениями. При увеличении Delta округление становится грубее, что может

влиять на вычисление очередного приближения, так как в формуле Ньютона используются одновременно $f(x_n)$ и $f'(x_n)$.

Исследование проводится следующим образом:

1. Вычисляется опорное (эталонное) значение корня x_{ref} при $\Delta = 0$ и высокой точности (например, $Eps = 10^{-14}$).
2. Далее метод NEWTON запускается при фиксированной точности (например, $Eps = 10^{-6}$), но при различных значениях Δ (например, 0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001).
3. Для каждого Δ фиксируются найденное значение корня x_{Δ} и число итераций $Iterations$.
4. Рассчитывается абсолютное отклонение от эталона: $AbsErrorToRef = |x_{\Delta} - x_{ref}|$.

Результаты записываются в файл:

sensitivity_vs_delta_newton.csv со столбцами: Δ ; Eps ; $Iterations$; $Root$; $AbsErrorToRef$.

Далее по этим данным строится график зависимости $AbsErrorToRef$ от Δ . По графику делается вывод о том, как изменение точности округления Δ влияет на точность найденного корня: при уменьшении Δ найденный корень приближается к x_{ref} и ошибка $AbsErrorToRef$ уменьшается; при больших Δ ошибка увеличивается, что демонстрирует чувствительность метода Ньютона к погрешностям в вычислении $f(x)$ и $f'(x)$.

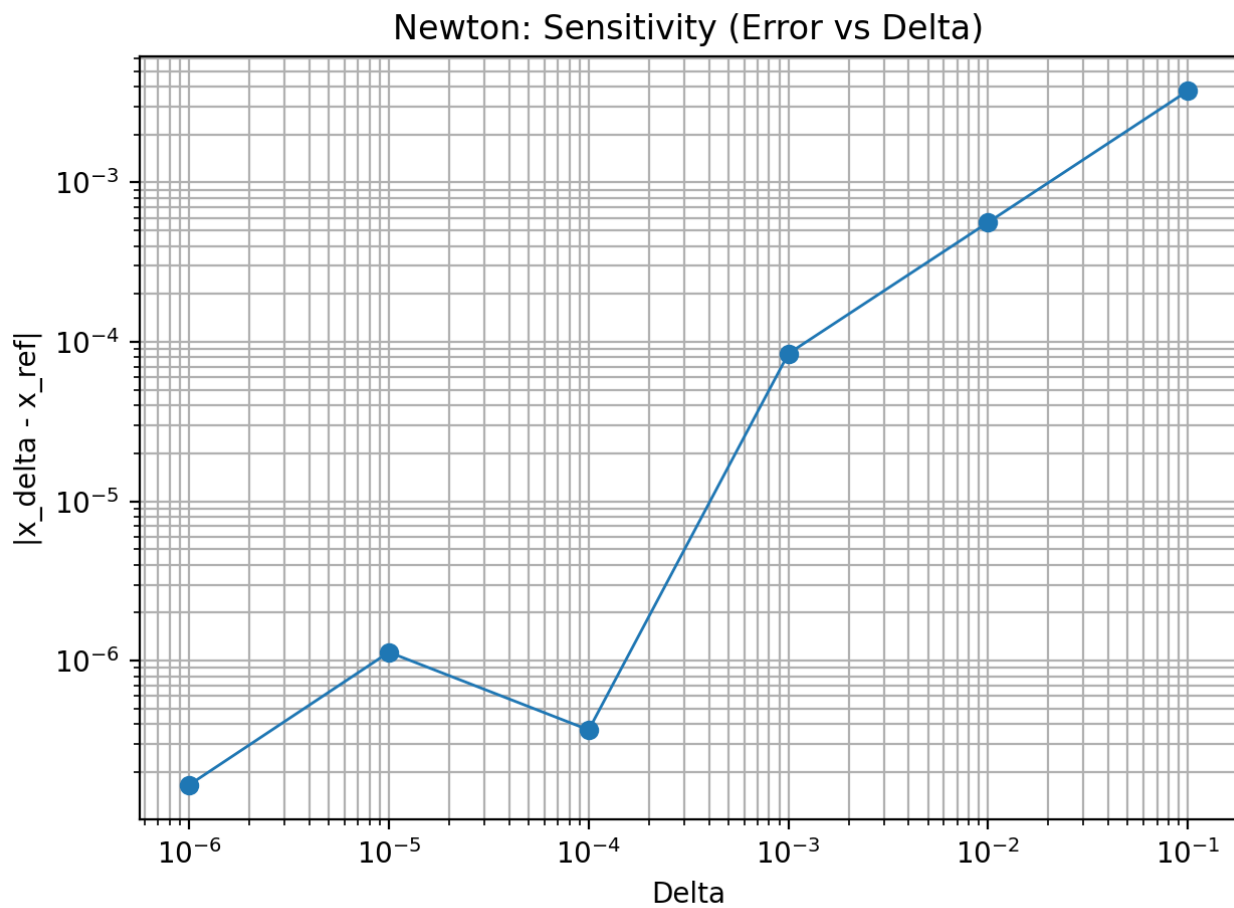


Рисунок 2 — Зависимость абсолютной ошибки $|x_{\text{Delta}} - x_{\text{ref}}|$ от Δ (чувствительность метода Ньютона к округлению).

Вывод.

В ходе лабораторной работы был изучен метод Ньютона для численного решения нелинейного уравнения $f(x) = 0$ на примере функции $f(x) = \arcsin(2x / (1 + x^2)) - e^{-x^2}$. Был выполнен этап отделения корня на интервале $[0; 0.5]$, составлены подпрограммы вычисления $f(x)$ и $f'(x)$, а также реализованы процедуры Round и NEWTON. Для обеспечения устойчивой сходимости было выбрано начальное приближение x_0 из $[Left; Right]$ по условию $f(x_0) * f'(x_0) > 0$, что позволило запустить итерационный процесс с корректным направлением приближения к корню. Проведены вычисления при различных значениях точности Eps в диапазоне от 0.1 до 0.000001 и исследована скорость сходимости метода, результаты представлены в виде таблицы и графика зависимости числа итераций от Eps. Также исследована чувствительность метода к ошибкам исходных данных: при увеличении Delta округление значений $f(x)$ и $f'(x)$ сильнее влияет на вычисление очередных приближений и может увеличивать отклонение от опорного значения, а при уменьшении Delta найденный корень приближается к эталонному, что подтверждает работоспособность метода при достаточно малых погрешностях вычислений.

ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: methods.h

```
#pragma once
#include <functional>

struct NewtonResult {
    double x;          // найденное приближение корня
    int iters;         // число итераций
    bool ok;           // сошлось ли
    double last_step; //  $|x_n - x_{n-1}|$ 
};

double Round(double value, double Delta);

// Метод Ньютона:
// f - функция
// fp - производная f'(x)
// Left, Right - интервал, где изолирован корень (для оценки m1 и M2)
// x0 - начальное приближение (выбирается так, чтобы  $f(x_0) * f'(x_0) > 0$ )
// Eps - требуемая точность
// nmax - ограничение итераций
// Delta - округление значений f и f' через Round (0 => без округления)
NewtonResult NEWTON(const std::function<double(double)>& f,
                    const std::function<double(double)>& fp,
                    double Left, double Right,
                    double x0,
                    double Eps,
                    int nmax = 1000000,
                    double Delta = 0.0);
```

Название файла: methods.cpp

```
#include "methods.h"
#include <cmath>
#include <limits>
#include <algorithm>

// Округление к ближайшему кратному Delta
double Round(double value, double Delta) {
    if (Delta <= 0.0) return value;
    return std::round(value / Delta) * Delta;
}

// Численная вторая производная (центральная разность)
// Нужна для оценки M2 и для выбора x0 (условие  $f(x_0) * f'(x_0) > 0$ )
static double second_derivative_num(const std::function<double(double)>& f, double x) {
    double h = 1e-5 * std::max(1.0, std::abs(x));
    double f1 = f(x - h);
    double f2 = f(x);
    double f3 = f(x + h);
    return (f3 - 2.0 * f2 + f1) / (h * h);
}

NewtonResult NEWTON(const std::function<double(double)>& f,
                    const std::function<double(double)>& fp,
```

```

        double Left, double Right,
        double x0,
        double Eps,
        int nmax,
        double Delta) {
NewtonResult res{};
res.x = std::numeric_limits<double>::quiet_NaN();
res.iters = 0;
res.ok = false;
res.last_step = std::numeric_limits<double>::infinity();

// Оценка  $m1 = \min |f'|$  и  $M2 = \max |f''|$  на  $[Left; Right]$  для критерия
(3.1)
double m1 = std::numeric_limits<double>::infinity();
double M2 = 0.0;

const int SAMPLES = 1000;
for (int i = 0; i <= SAMPLES; ++i) {
    double x = Left + (Right - Left) * (double)i / (double)SAMPLES;
    double d1 = std::abs(fp(x));
    double d2 = std::abs(second_derivative_num(f, x));
    if (d1 < m1) m1 = d1;
    if (d2 > M2) M2 = d2;
}

//  $eps0 = 2*m1/M2 * Eps$  (формула (3.1))
double eps0 = Eps;
if (m1 > 0.0 && std::isfinite(m1) && M2 > 0.0 && std::isfinite(M2)) {
    eps0 = (2.0 * m1 / M2) * Eps;
}

double x_prev = x0;
double fx_prev = Round(f(x_prev), Delta);
double fpx_prev = Round(fp(x_prev), Delta);

if (std::abs(fpx_prev) < 1e-15) {
    return res; // производная почти ноль => деление опасно
}

for (int n = 1; n <= nmax; ++n) {
    double x_next = x_prev - fx_prev / fpx_prev;
    double step = std::abs(x_next - x_prev);

    double fx_next = Round(f(x_next), Delta);
    double fpx_next = Round(fp(x_next), Delta);

    res.last_step = step;

    // Основной критерий из методички:  $|x_n - x_{n-1}| < eps0$ 
    if (step < eps0) {
        res.x = x_next;
        res.iters = n;
        res.ok = true;
        return res;
    }

    // Дополнительный контроль (не мешает, но полезен при больших
Delta)

```

```

        if (std::abs(fx_next) < Eps) {
            res.x = x_next;
            res.iters = n;
            res.ok = true;
            return res;
        }

        if (std::abs(fpx_next) < 1e-15) {
            // дальше делить нельзя
            res.x = x_next;
            res.iters = n;
            res.ok = false;
            return res;
        }

        x_prev = x_next;
        fx_prev = fx_next;
        fpx_prev = fpx_next;
    }

    res.x = x_prev;
    res.iters = nmax;
    res.ok = false;
    return res;
}

```

Название файла: main.cpp

```

#include <iostream>
#include <iomanip>
#include <cmath>
#include <vector>
#include <fstream>
#include "methods.h"

// f(x) = arcsin(2x/(1+x^2)) - e^(-x^2)
double f(double x) {
    double t = (2.0 * x) / (1.0 + x * x);

    // защита от погрешностей double: зажимаем в [-1;1]
    if (t > 1.0) t = 1.0;
    if (t < -1.0) t = -1.0;

    return std::asin(t) - std::exp(-x * x);
}

// f'(x) = t'(x)/sqrt(1-t(x)^2) + 2x*e^(-x^2)
// t(x) = 2x/(1+x^2)
// t'(x) = 2(1 - x^2)/(1 + x^2)^2
double fp(double x) {
    double denom = (1.0 + x * x);
    double t = (2.0 * x) / denom;

    if (t > 1.0) t = 1.0;
    if (t < -1.0) t = -1.0;

    double tp = 2.0 * (1.0 - x * x) / (denom * denom);
    double s = std::sqrt(std::max(0.0, 1.0 - t * t));
}

```

```

// если s очень мал (рядом с t=±1), защищаемся
double part1 = (s < 1e-15) ? 0.0 : (tp / s);
double part2 = 2.0 * x * std::exp(-x * x);

return part1 + part2;
}

// Численная f'(x) для выбора x0 по условию f(x0)*f'(x0)>0
double f2_num(double x) {
    double h = 1e-5 * std::max(1.0, std::abs(x));
    return (f(x + h) - 2.0 * f(x) + f(x - h)) / (h * h);
}

// Авто-отделение корня: ищем смену знака на сетке
bool find_bracket(double x_min, double x_max, double step, double &Left,
double &Right) {
    double prev = x_min;
    double fprev = f(prev);

    for (double x = x_min + step; x <= x_max; x += step) {
        double fx = f(x);

        if (fprev == 0.0) { Left = prev; Right = prev; return true; }
        if (fprev * fx < 0.0) { Left = prev; Right = x; return true; }

        prev = x;
        fprev = fx;
    }
    return false;
}

int main() {
    std::cout << std::fixed << std::setprecision(12);

    // 1) Отделяем корень (это только диапазон поиска!)
    double Left = 0.0, Right = 0.0;
    if (!find_bracket(-5.0, 5.0, 0.1, Left, Right)) {
        std::cerr << "Не удалось отделить корень на [-5;5] с шагом 0.1\n";
        return 1;
    }

    std::cout << "Отделение корня (авто): [" << Left << "; " << Right <<
    "]" << "\n";
    std::cout << "f(Left)=" << f(Left) << ", f(Right)=" << f(Right) <<
    "\n";

    // 2) Выбор x0: требуется f(x0)*f'(x0) > 0
    double x0 = 0.5 * (Left + Right);
    double condL = f(Left) * f2_num(Left);
    double condR = f(Right) * f2_num(Right);
    double condM = f(x0) * f2_num(x0);

    if (condL > 0.0) x0 = Left;
    else if (condR > 0.0) x0 = Right;
    else if (condM > 0.0) x0 = 0.5 * (Left + Right);
    // если ни одна точка не подошла (редко), оставляем середину

    std::cout << "Выбрано начальное приближение x0=" << x0

```

```

        << " (проверка  $f(x_0) * f'(x_0) > 0$ )\n\n";

// 3) Опорное значение корня (эталон): Delta=0, очень высокая точность
auto ref = NEWTON(f, fp, Left, Right, x0, 1e-14, 1000000, 0.0);
if (!ref.ok) {
    std::cerr << "Не удалось получить reference-корень методом
Ньютона\n";
    return 2;
}
std::cout << "Reference root (Eps=1e-14, Delta=0): x=" << ref.x
    << "  iters=" << ref.iters << "\n\n";

// 4) Скорость сходимости: Iterations vs Eps (0.1 .. 1e-6)
std::ofstream feps("iterations_vs_eps_newton.csv");
feps << "Eps;Iterations;Root;LastStep\n";

for (int i = 0; i <= 50; ++i) {
    double t = i / 50.0;          // 0..1
    double p = -1.0 - 5.0 * t;    // -1..-6
    double Eps = std::pow(10.0, p);

    auto r = NEWTON(f, fp, Left, Right, x0, Eps, 1000000, 0.0);

    feps << std::setprecision(16)
        << Eps << ";" << r.iters << ";" << r.x << ";" << r.last_step
<< "\n";
}
feps.close();
std::cout << "CSV сохранён: iterations_vs_eps_newton.csv\n";

// 5) Обусловленность: влияние Delta (округление f и f')
const double EpsFix = 1e-6;
std::vector<double> deltas = {0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-
6};

std::ofstream fdel("sensitivity_vs_delta_newton.csv");
fdel << "Delta;Eps;Iterations;Root;AbsErrorToRef\n";

for (double Delta : deltas) {
    auto r = NEWTON(f, fp, Left, Right, x0, EpsFix, 1000000, Delta);
    double err = std::abs(r.x - ref.x);

    fdel << std::setprecision(16)
        << Delta << ";" << EpsFix << ";" << r.iters << ";" << r.x <<
";" << err << "\n";
}
fdel.close();
std::cout << "CSV сохранён: sensitivity_vs_delta_newton.csv\n";

return 0;
}

```

Название файла: plot_graphs.py

```

import pandas as pd
import matplotlib.pyplot as plt

# 1) Iterations vs Eps

```

```

eps_df = pd.read_csv("iterations_vs_eps_newton.csv", sep=";")

plt.figure()
plt.plot(eps_df["Eps"], eps_df["Iterations"], marker="o", linewidth=1)
plt.xscale("log")
plt.xlabel("Eps")
plt.ylabel("Iterations")
plt.title("Newton: Iterations vs Eps")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("iterations_vs_eps_newton.png", dpi=200)
plt.show()

# 2) Error vs Delta (Delta=0 убираем для log по X)
delta_df = pd.read_csv("sensitivity_vs_delta_newton.csv", sep=";")
delta_df_nonzero = delta_df[delta_df["Delta"] > 0].copy()

plt.figure()
plt.plot(delta_df_nonzero["Delta"], delta_df_nonzero["AbsErrorToRef"],
marker="o", linewidth=1)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Delta")
plt.ylabel("|x_delta - x_ref|")
plt.title("Newton: Sensitivity (Error vs Delta)")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("sensitivity_vs_delta_newton.png", dpi=200)
plt.show()

```